Software Engineering Department
Braude College of Engineering

Final Project – Phase A (61998)

Horizon Forecast

Satellite Cloud IR Nowcasting Transformer Model

CODE: 26-1-R-1

By:

Gilad Boudman
Or Mordechay Hod

Advisors:

Mrs. Elena Kramer
Dr. Dan Lemberg

Link to GitHub:

https://github.com/Horizon-Forecast/Satellite-Cloud-IR-Nowcasting-Transformer-Model

Contents

Abstract

Climate change has intensified the frequency and severity of extreme weather events, creating an urgent need for precise, real-time nowcasting systems. Traditional Numerical Weather Prediction (NWP) models suffer from significant latency, while standard Deep Learning approaches often fail to capture the physical consistency between atmospheric drivers (Wind, Temperature) and surface precipitation. A critical challenge remains the ”Sparse Supervision Gap”: while satellite imagery offers dense global coverage, ground truth data for rain and hail are available only at scattered station points.

This project introduces Horizon Forecast, a Cascaded Dual-Supervision Network designed to bridge this gap. Our architecture integrates the computational efficiency of the SimVPv2 video processor with the probabilistic reasoning of a transformer-based decision head. The system operates in a physics-informed two-stage cascade: First, a Visual-Physical Head predicts the thermodynamic drivers (Surface Wind and Temperature) and the future cloud structure, verified against dense satellite imagery to ensure spatial consistency. Second, these verified atmospheric states serve as the conditioning context for a Precipitation Head that infers the intensity of specific storm events.

The model will be trained and evaluated on high-resolution SEVIRI satellite imagery and IMS ground station data focused on the complex climatic region of Israel. By explicitly modeling the dependency of precipitation on its physical drivers, Horizon Forecast generates a synchronized 4-channel predictive map comprising one visual verification layer (clouds) and three distinct physical variables Rain Intensity, Wind Speed, and Surface Temperature-offering a scalable, physics-aware solution for modern meteorological challenges.

Keywords: Next-Frame Prediction, Cascaded Dual-Supervision, SimVPv2, Physics-Informed AI, Weather Nowcasting, Multi-Variable Forecasting.

1 Introduction

1.1 Background and Motivation

Global climate change has significantly increased the frequency and severity of extreme weather events, such as heatwaves, intense rainfall, and storms [6]. For meteorological agencies, the ability to predict these events with high precision is critical for disaster mitigation. Traditionally, weather prediction relies on NWP models, which simulate atmospheric dynamics by solving complex partial differential equations (PDEs) [1].

However, NWP models suffer from critical limitations in the nowcasting (0-4 hour) regime. First, solving high-dimensional PDEs is computationally demanding, often requiring supercomputers and resulting in significant latency that renders them too slow for real-time alerts [1]. Second, these models rely on parameterization schemes for unresolved processes, which can degrade performance at fine spatiotemporal scales [9]. As noted in recent literature, global NWP models are often only available at hourly intervals, making them inappropriate sources of guidance for rapidly evolving hazards [3].

In response to these challenges, Deep Learning (DL) has emerged as a powerful alternative, capable of learning intricate dependencies directly from data. Since the introduction of the ConvLSTM [9], the field has evolved into two distinct architectural paradigms. On one hand, Convolutional Neural Networks (CNNs), such as SimVPv2, have demonstrated that streamlined architectures using Gated Spatiotemporal Attention (gSTA) can achieve state-of-the-art performance with superior computational efficiency [10]. On the other hand, Vision Transformers, such as SaTformer, have introduced full space-time attention mechanisms that effectively capture global context and extreme events, although with higher computational costs [3].

The motivation for this research lies in the ”gap” between these two approaches. While SimVPv2 offers the efficiency required for real-time operation [10], it may lack the probabilistic fidelity needed for extreme event detection. Conversely, SaTformer provides robust uncertainty estimation through classification-based bucket prediction [3], but scales quadratically. This project aims to bridge this divide by proposing a ”Hybrid Cascade” model that leverages the efficiency of SimVPv2 as a backbone while incorporating the probabilistic reasoning of SaTformer to deliver high-fidelity, physics-informed nowcasts.

1.2 Problem Statement

Despite the rapid advancement of Deep Learning in meteorology, current state-of-the-art architectures face two fundamental limitations that prevent them from achieving reliable, high-fidelity nowcasting.

1. The ”Blurriness” of Statistical Averaging
The majority of existing video prediction models, including the widely used SimVPv2, rely on regression-based loss functions such as Mean Squared Error (MSE) [10]. While this approach effectively minimizes the overall error rate, it operates on a unimodal assumption that averages all possible future outcomes. As noted by Harris et al. [3], minimizing MSE inherently smooths out high-frequency details. This statistical averaging results in ”blurry” predictions that systematically under-represent extreme events, causing models to struggle with the sharp boundaries and high intensities associated with severe hazards like hail cores or flash floods.

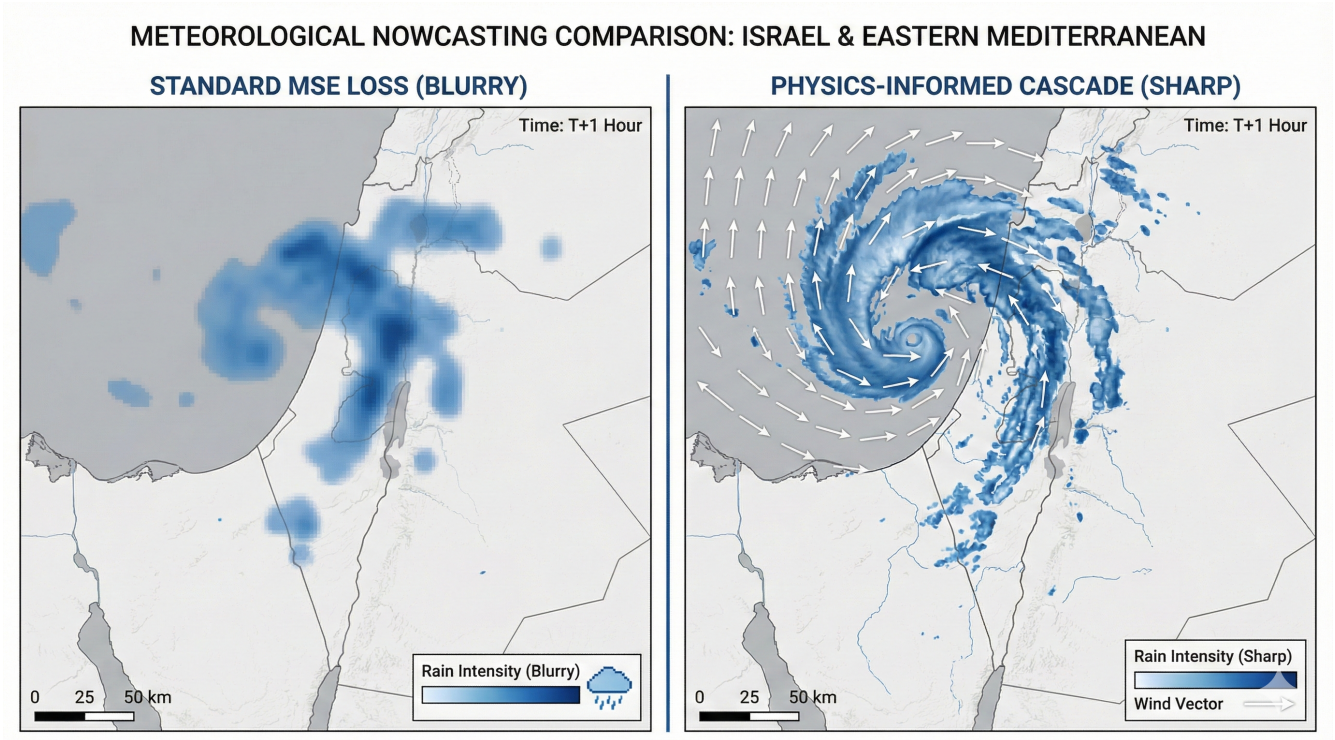

Figure 1: **Conceptual Illustration.** An idealized comparison showing the "Physics-Gap." Standard MSE loss typically yields blurry predictions (Left), while our proposed architecture aims to achieve the sharp structural definition depicted on the Right. *(Image generated using Google Nano Banana Pro).*

2. Physics-Blind Inference

Furthermore, a critical context gap exists in how standard deep learning models interpret meteorological data. Architectures like SimVPv2 treat precipitation forecasting as a video-to-video translation task, prioritizing the minimization of pixel-level error (optical flow) over the preservation of atmospheric dynamics. Research indicates that these models are effectively "physics-blind", as they lack the explicit inductive biases required to model the fundamental physical laws—such as conservation of mass and momentum that drive weather systems [8]. Consequently, by ignoring these governing equations, such models fail to capture the causal reasoning necessary to predict the non-linear evolution of storm intensity and direction.

1.3 Project Goals

The primary objective of this research is to investigate the efficacy of a "Physics-Driven Cascade" architecture for precipitation nowcasting. Specifically, this project aims to research whether explicitly modeling the thermodynamic drivers (Wind and Temperature) in a first-stage cascade significantly improves the prediction fidelity of the storm manifestation (Clouds and Rain) in the second stage.

The specific research goals are:

1. **Research the "Driver-First" Cascade:** To design a sequential neural network where the model first predicts the Thermodynamic Drivers (Surface Wind and Temperature). This creates a "physics-informed" latent state, effectively testing the hypothesis that the model must understand why the storm is moving (Wind/Temp) before it can predict where it is going.

2. **Predict Cloud Structure and Rain Intensity:** To utilize the predicted Wind and Temperature maps from Stage 1 as the conditioning context for Stage 2. In this second stage, the model generates the Visual Cloud Map (verified by dense satellite imagery) and the Rain Intensity Map (verified by IMS stations), ensuring that the storm's visual movement is consistent with its physical drivers.

3. **Multi-Variable Forecasting:** To extend the scope of nowcasting beyond simple rainfall intensity. The system is designed to generate a synchronized 4-channel output tensor - comprising Cloud Visual, Wind Speed, Surface Temperature, Rain Intensity providing a comprehensive view of storm evolution.

2 Literature Review

2.1 Spatiotemporal Predictive Learning

Spatiotemporal predictive learning is a foundational paradigm in meteorological forecasting, aiming to generate future frame sequences based on historical observations by modeling both spatial deformations and temporal evolution [10]. Traditionally, Numerical Weather Prediction (NWP) models have served as the bedrock of atmospheric science, solving complex Partial Differential Equations (PDEs) to resolve atmospheric dynamics [1]. However, these physics-based models are computationally demanding and often suffer from performance degradation at nowcasting timescales (0-4 hours) due to spin-up time and resolution constraints [3, 8].

To address these limitations, Deep Learning (DL) approaches have emerged as powerful alternatives. Early data-driven methods were dominated by Recurrent Neural Networks (RNNs). The seminal ConvLSTM architecture extended fully connected Long Short Term Memory (LSTM) with convolutional structures to capture spatiotemporal correlations simultaneously [9, 8]. Subsequent innovations, such as PredRNN, introduced Spatiotemporal LSTM (ST-LSTM) units with zigzag memory flows to better preserve non-stationary variations [10]. Despite their success, RNN-based models face inherent efficiency bottlenecks: their sequential processing nature prevents parallelization, leading to high training latency and slow inference speeds, which are critical drawbacks for real-time forecasting systems [10].

2.2 Convolutional Approaches (SimVPv2)

In response to the computational inefficiencies of recurrent architectures, recent research has pivoted toward fully convolutional frameworks that treat predictive learning as a direct tensor-to-tensor mapping problem. The SimVP model established a strong baseline by utilizing a pure "CNN-CNN-CNN" architecture-comprising a spatial encoder, a translator, and a spatial decoder-thereby eliminating recurrent units entirely [10].

Building on this foundation, SimVPv2 further streamlines the architecture by removing the heavy U-Net-like hierarchical structures used in the original SimVP translator. Instead, it introduces a novel Gated Spatiotemporal Attention (gSTA) mechanism [10]. The gSTA module is designed to capture long-range dependencies without the massive computational cost of Transformers. It achieves this by decomposing large-kernel convolutions into three efficient components: depth-wise convolutions for local receptive fields, depth-wise dilation convolutions for distant connections, and 1×1 convolutions for channel mixing [10]. This "attention-via-convolution" approach allows the model to adaptively filter informative features

while significantly reducing Floating Point Operations (FLOPs) and inference time compared to both RNNs and complex Vision Transformers [10].

2.3 Transformer Approaches (SaTformer)

While CNNs excel at local feature extraction, Transformers have become the state-of-the-art for modeling global dependencies. In the specific domain of precipitation forecasting, standard regression approaches often fail to capture the multi-modal and heavy-tailed distribution of rainfall data [3]. The SaTformer architecture addresses this by reformulating precipitation forecasting as a classification problem rather than simple regression. It projects continuous rainfall values into discrete probability buckets (e.g., 64 classes), allowing the model to learn a probability distribution over possible outcomes [3].

Architecturally, SaTformer distinguishes itself by employing Full Space-Time Self-Attention rather than separating spatial and temporal attention steps. Because the input satellite patches are spatially compact, SaTformer can computationally afford to let every token attend to every other token across both space and time simultaneously [3]. This global context is crucial for detecting the formation and movement of storm clusters [6]. Furthermore, to combat the extreme class imbalance inherent in weather data (where ”no rain” is the dominant class), SaTformer utilizes a Class-Weighted Cross-Entropy Loss, which penalizes errors on rare, extreme rainfall events more heavily than on common weather states [3].

2.4 Gap Analysis

Despite the significant advancements in spatiotemporal predictive learning, a critical gap remains in effectively balancing computational efficiency with the ability to model complex, global atmospheric dynamics.

First, while fully convolutional architectures like SimVPv2 achieve state-of-the-art efficiency by eliminating recurrent units [10], they inherently prioritize local spatial features. Although the Gated Spatiotemporal Attention (gSTA) mechanism expands the receptive field, it may still struggle to capture the full global context required for complex, multi-scale phenomena like storm cluster formation, which Transformers excel at [3].

Conversely, pure Transformer-based approaches such as SaTformer demonstrate superior capability in modeling global dependencies and handling long-tailed distributions via classification [3]. However, applying full space-time self-attention to high-resolution data is computationally expensive, often limiting these models to smaller spatial patches or requiring significant hardware resources [8].

Furthermore, existing literature often treats different weather variables (e.g., wind, temperature, precipitation) with uniform architectural inductive biases. There is a lack of hybrid frameworks that explicitly decouple the ”physics-driven” variables (wind, temperature) - which follow clear local gradients suitable for CNNs - from the ”event-driven” variables (precipitation)-which require global attention to detect discrete, extreme events [6].

Therefore, a unified architecture that integrates the efficiency of a CNN-based extractor (SimVPv2) for spatiotemporal feature encoding with the global reasoning capabilities of a Transformer head for final precipitation classification remains an underexplored avenue. This project seeks to bridge this gap by proposing a hybrid ”SimVPv2 Body + Transformer Head” architecture.

3 Dataset and Data Analysis

To develop a region-specific nowcasting system for the Eastern Mediterranean (Israel), this research constructs a custom "Hybrid" dataset. This dataset fuses high-resolution geostationary satellite imagery with dense ground-based meteorological observations and static topographic maps.

3.1 Data Sources

The dataset is constructed from three synchronized streams, representing the "Input" (Past Atmospheric State), the "Dual-Targets" (Future Drivers & Manifestation), and the "Alignment Metadata" required to bridge them:

- **Input A: Satellite Imagery (EUMETSAT):** The primary visual input is acquired from the Meteosat Second Generation (MSG) satellites via the EUMETSAT Data Store [2]. We utilize High Rate SEVIRI Level 1.5 data focused on the Eastern Mediterranean. Two spectral channels are selected for feature extraction:
  - **IR 10.8μm (Thermal Infrared):** Critical for estimating cloud-top height and identifying convective storm structures even during night time.
  - **WV 6.2μm (Water Vapor):** Captures upper-tropospheric moisture flow, providing the model with context on high-altitude wind steering currents.
- **Input B: Static Geospatial Features (Topography):** Since precipitation in the Eastern Mediterranean is strongly modulated by mountain structure (e.g., the Judean Mountains forcing air upwards), we incorporate a Digital Elevation Model (DEM) derived from the NASA SRTM dataset. This static 256×256 height map is concatenated to every temporal frame of the input sequence.
- **Target A: Thermodynamic Drivers (Stage 1 Ground Truth):** The first prediction stage is verified against physical station measurements to ensure the model understands atmospheric dynamics before generating visuals. These variables are sparse (station-based):
  - Surface Temperature ($^{\circ}C$)
  - Surface Wind Velocity (m/s)
  - Wind Direction ($^{\circ}$)
- **Target B: Visual Cloud Structure (Stage 2 Ground Truth):** Once the drivers are established, the model predicts the visual evolution. We utilize Future Satellite Frames (e.g., T_{+15min}) as a dense ground truth target. This allows the model to calculate error over the entire 256×256 grid, ensuring the predicted cloud movement is consistent with the wind vectors from Target A.
- **Target C: Precipitation Impact (Stage 3 Ground Truth):** The final stage predicts the ground-level impact, conditioned on both the physical drivers (A) and the cloud structure (B). These variables are derived from the IMS network:
  - Precipitation Intensity (mm/hr)

- **Geospatial Alignment Mask (Pre-Computed):** To enable learning from sparse data without "hallucinating" zeros in empty regions, we generate a static Station Coordinate Mask. This is a sparse 256×256 matrix where pixels corresponding to active IMS stations are marked with indices.

  This artifact functions as a masking layer for Sparse Supervision. It ensures that gradients are backpropagated only from active station points, preventing the model from learning zeros in empty regions. Through convolutional diffusion, the network learns to bridge these geospatial gaps, effectively transforming scattered point data into a dense, country-wide meteorological tensor.

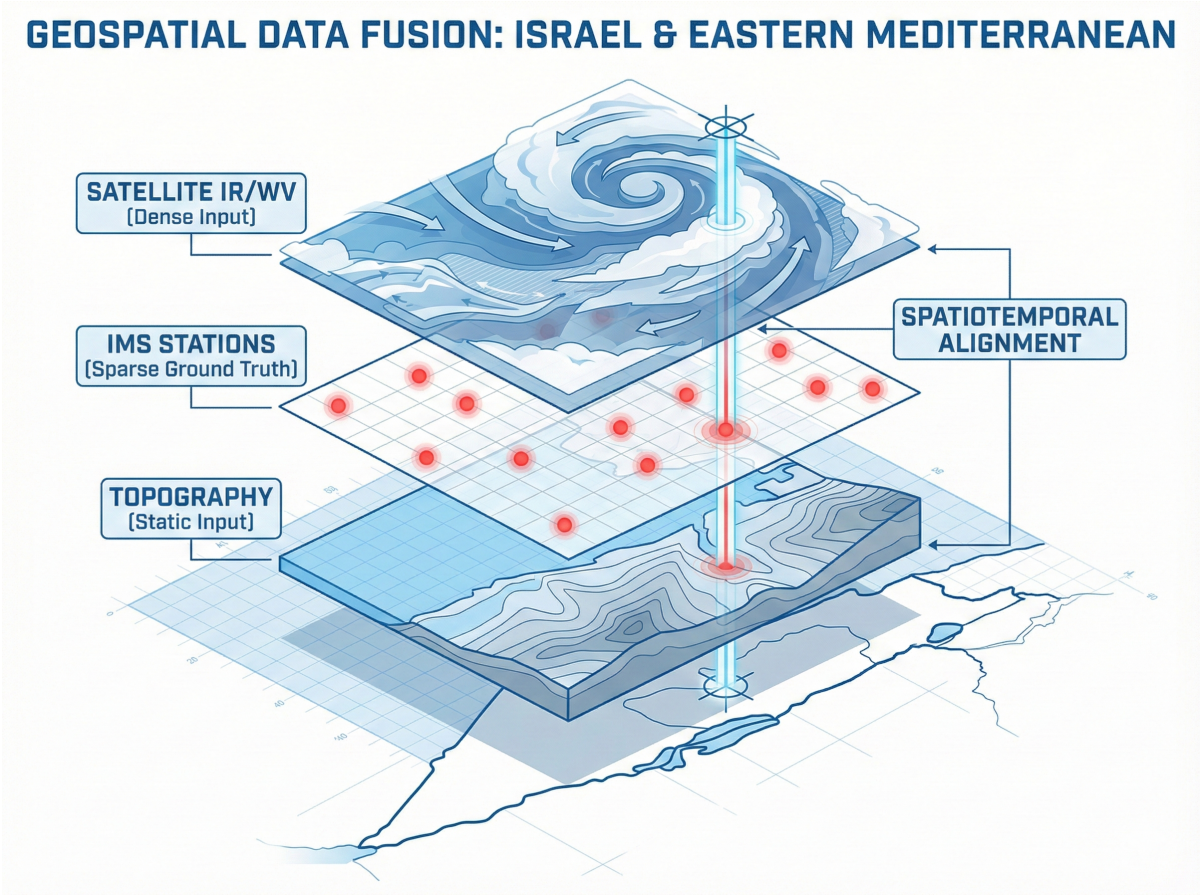

Figure 2: **Data Fusion Strategy.** A breakdown of the model's multi-modal input tensor. The system aligns three distinct geospatial layers: static topography (Bottom), sparse point-based station measurements (Middle), and dense satellite imagery (Top) onto a unified 256×256 grid over the Israeli domain. The vertical line illustrates the precise spatiotemporal synchronization required for valid training. *(Image generated using Google Nano Banana Pro).*

3.1.1 Dataset Specifications and Partitioning

To ensure the model is evaluated on its ability to generalize to future unseen weather, the dataset is strictly partitioned chronologically rather than randomly to prevent temporal data leakage.

- **Spatial Domain:** The region of interest is defined by the bounding box $[29.0°N, 34.0°N] \times [34.0°E, 36.0°E]$, covering Israel and the Eastern Mediterranean marine approach.

- **Temporal Scope:** The dataset spans **5 years** (Jan 2020 – Dec 2025).
- **Data Splitting:**
  - **Training (2020–2023):** Optimization of weights.
  - **Validation (2024 Jan–Jun):** Hyperparameter tuning.
  - **Test (2024 Jul – 2025 Dec):** Strictly held-out for final benchmarking.

4 Project Requirements

4.1 Functional Requirements

The proposed system acts as an end-to-end Deep Learning pipeline. The functional requirements are categorized by the stage of the machine learning lifecycle:

Data Ingestion and Preprocessing

- **FR-01 Temporal Data Synchronization:** The system shall transform continuous variable streams (Temperature, Wind, Precipitation) into discrete 15-minute intervals ($XX:00, XX:15, XX:30, XX:45$).
- **FR-02 Geospatial Station Mapping:** The system shall map the GPS coordinates of each active IMS station to a corresponding (x, y) pixel index on the satellite imagery grid.
- **FR-03 Multi-Channel Input Aggregation:** The system shall combine dynamic satellite imagery (IR 10.8μm, WV 6.2μm) and static Digital Elevation Model (DEM) data to create the unified model input for each time step.
- **FR-04 Data Normalization:** The system shall normalize all input features and target variables to a uniform numerical range prior to processing.

Model Architecture and Training

- **FR-05 Forecast Map Generation:** The system shall generate a predictive weather map for the target region, consisting of three synchronized layers:
  1. **Thermodynamics:** Surface Wind Speed and Temperature.
  2. **Visuals:** Cloud Structure.
  3. **Precipitation:** Rain Intensity.
- **FR-06 Dual-Source Validation:** The system shall validate model predictions by computing errors against two distinct ground truth sources:
  - **Visual Verification:** Comparison against future satellite imagery across the entire geospatial domain.

  - **Physical Verification:** Comparison against actual IMS station measurements at their specific coordinates.

- **FR-07 Automated Model Persistency:** The system shall automatically save the model's training progress at the completion of each training cycle. Additionally, it shall identify and isolate the specific model version that yields the highest validation performance.
- **FR-08 Training Performance Monitoring:** The system shall record real-time performance metrics throughout the training process, including training loss, validation loss, and optimization parameters.
- **FR-09 Deterministic Initialization:** The system shall allow the enforcement of deterministic behavior to ensure the exact reproducibility of experimental results.

Evaluation and Output

- **FR-10 Chronological Data Partitioning:** The system shall partition the dataset into training, validation, and testing subsets based on chronological order to ensure temporal separation and prevent data leakage.
- **FR-11 Verification Metric Calculation:** The system shall compute and report meteorological verification metrics, including Root Mean Square Error (RMSE) for continuous parameters and Critical Success Index (CSI) for event classification.

4.2 Non-Functional Requirements

These requirements define the system's operational constraints, performance benchmarks, and quality attributes necessary for a production-grade forecasting tool.

Performance and Efficiency

- **NFR-01 Inference Latency:** The trained model shall generate a complete forecast sequence with low latency, ensuring that the inference time is significantly shorter than the data acquisition interval to satisfy real-time alerting.
- **NFR-02 GPU Memory Footprint:** The complete inference pipeline, comprising model weights, buffers, and input tensors, shall be optimized to fit within the VRAM of standard consumer-grade GPUs. This ensures deployment feasibility without requiring high-end data center hardware.
- **NFR-03 Resource Efficiency:** The training pipeline shall be optimized for GPU saturation, ensuring that Data I/O bottlenecks do not cause GPU utilization to drop below 80% during active training epochs.

Reliability and Robustness

- **NFR-04 Numerical Stability:** The system shall ensure operational continuity and output validity by automatically handling numerical computation errors, preventing system failure or the propagation of invalid results.

Scalability and Maintainability

- **NFR-05 Temporal Scalability:** The system shall support the processing of expanding historical datasets without requiring architectural modification or incurring significant performance degradation.
- **NFR-06 Architectural Modularity:** The system shall maintain a modular separation between data processing and predictive modeling components to facilitate the seamless replacement or upgrade of the prediction algorithms.

5 Research Methodology

5.1 Development Process

The project schedule is structured into three consecutive phases, moving from theoretical design to data engineering, and finally to model implementation and evaluation.

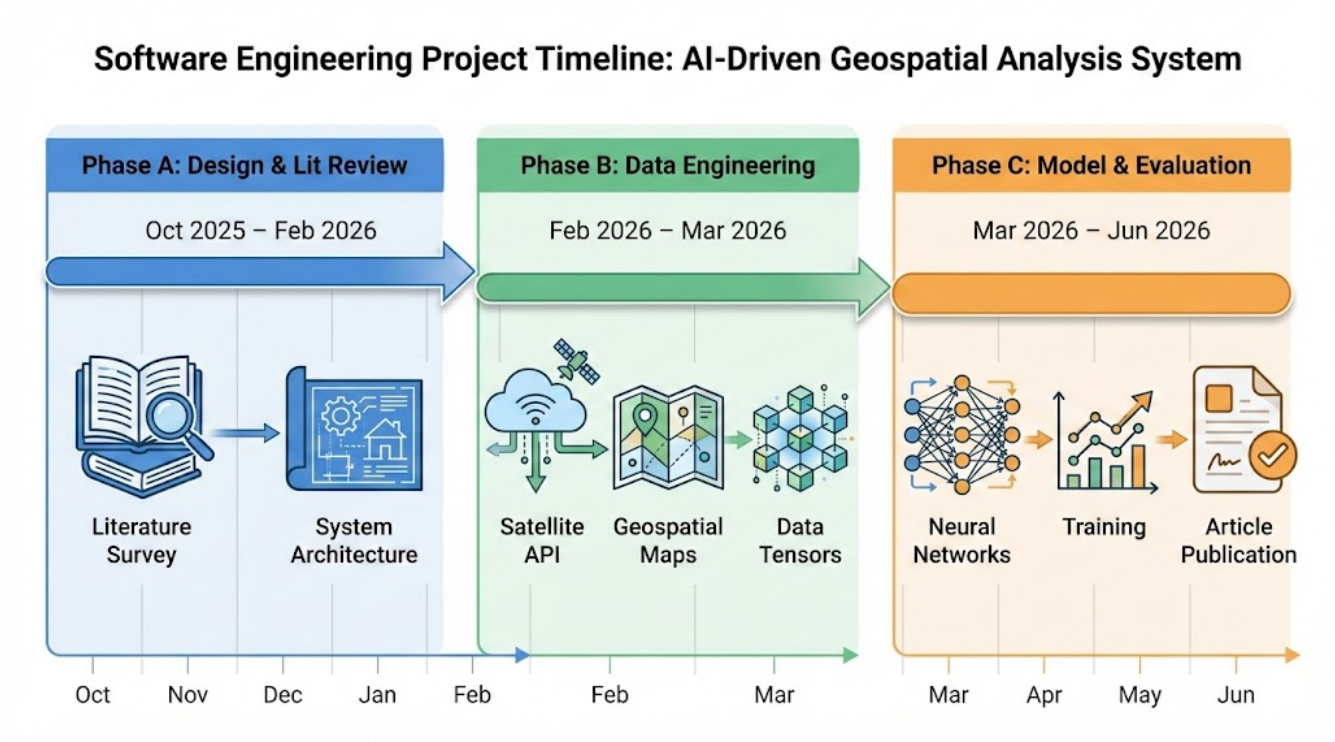

Figure 3: **Project Lifecycle.** The timeline is divided into Design, Data Engineering, and Model Development. *(Image generated using Google DeepMind Nano Banana Pro).*

Phase A: Literature Review and System Design (Oct 2025 – Feb 2026)

The initial phase focused on defining the theoretical scope and architectural blueprint.

- **Literature Survey:** Conducted a deep analysis of existing approaches, identifying the limitations of regression-based CNNs (SimVPv2) and the potential of Physics-Informed Deep Learning.
- **Architectural Design:** Defined the novel "Cascaded Dual-Supervision" structure, specifically designing the "Physics-First" cascade that predicts thermodynamic drivers before storm manifestation.
- **Documentation:** Compilation of the Phase A Proposal Book, establishing the mathematical formulation of the sparse supervision problem.

Phase B: Data Engineering and Pipeline Construction (Feb 2026 – Mar 2026)

Following the approval of the design, this phase focuses entirely on "getting the data" and building the ingestion infrastructure.

- **API Integration:** Implementation of Python scripts to authenticate and download bulk historical data from the EUMETSAT Data Store (Satellite) and IMS API (Ground Truth).
- **Geospatial Processing:** Processing the static datasets. This involves resizing the NASA SRTM Elevation Map (DEM) to 256×256 and generating the "Station Alignment Mask" that maps the exact GPS coordinates of IMS stations to pixel indices on the satellite grid.
- **Synchronization Logic:** Coding the linear interpolation algorithms required to align 10-minute ground station data with 15-minute satellite frames.
- **Tensor Construction:** Creating the automated pipeline that fuses the dynamic satellite frames, static topography, and aligned ground truth into the final (B, T, C, H, W) PyTorch tensors.

Phase C: Model Development and Evaluation (Mar 2026 – Jun 2026)

The final phase focuses on assembling the neural network, training it on the data, and proving its accuracy.

- **Model Assembly:** Programming the "Cascaded Dual-Supervision" architecture in Python. We will integrate the SimVPv2 backbone (for feature extraction) with the sequential output heads: Stage 1 for Thermodynamic Drivers (Wind/Temp) and Stage 2 for Storm Manifestation (Clouds/Rain).
- **Training and Tuning:** Feeding the historical dataset into the model to "teach" it weather patterns. We will run repeated experiments on the GPU, adjusting the hyperparameters (e.g., λ_{dense} vs λ_{sparse} loss weights) to balance physical consistency with visual accuracy.
- **Validation and Reporting:** Testing the final model on "unseen" data (from late 2024 and 2025). We will generate **side-by-side comparison maps** (Predicted vs. Actual) for the entire country to demonstrate the model's ability to interpolate between stations.

- **Final Documentation & Publication:** The project concludes with the compilation of the final engineering book and a scientific article. This documentation will formalize the hypothesis, detail the architectural implementation, and present the comparative results against standard benchmarks.

5.2 Tools and Technologies

The system is built using a modern Python technology stack, selected to handle large-scale weather data efficiently.

The Deep Learning Engine

- **PyTorch 2.1:** The primary library used to build the neural network. It handles the complex math behind the model's layers and allows the system to "learn" from data by adjusting its internal weights.
- **PyTorch Lightning:** A framework that organizes the PyTorch code. It automates the messy parts of training—like saving progress, logging errors, and running on the GPU—keeping our code clean and readable.

Data Engineering and Geospatial Processing

- **EUMDAC:** The official tool for downloading satellite data. It acts as a secure bridge to the European Space Agency's servers, allowing us to automatically fetch years of historical weather images.
- **Xarray & Dask:** Tools for handling "Big Data." Since our 5-year dataset is too large to fit into memory, these tools allow us to open and process the data in small chunks, streaming only what is needed for the current calculation.
- **Rasterio:** A mapping tool used to align different map projections. It reshapes the flat NASA topography maps to perfectly match the curved view of the weather satellite, ensuring that mountains and clouds align correctly on the grid.
- **Pandas:** The standard tool for data tables. We use it to clean the IMS station records, fix missing time gaps, and organize the ground truth data into a structured format.

Infrastructure

- **NVIDIA CUDA:** The software layer that allows our Python code to talk to the graphics card (GPU). This enables the massive parallel processing required to train the model in hours rather than weeks.
- **Git & GitHub:** Used for version control. It allows the team to save different versions of the code, track changes, and work on different features simultaneously without breaking the main system.

5.3 Expected Challenges and Mitigation Strategies

Developing a deep learning system on high-dimensional meteorological data presents several distinct risks. We have identified the critical failure modes and designed specific mitigation strategies for each.

1. The "Class Imbalance" Problem (Rare Events)

Challenge: Precipitation in the Eastern Mediterranean is a highly sporadic event, characterized by sharp climatological transitions. Our empirical analysis of IMS station data (2020–2025) reveals that measurable rainfall (> 0.1 mm) occurs in only **1.16%** of timestamps in Northern Israel, dropping to **0.24%** in the arid South [5].

This extreme sparsity is consistent with long-term climatological trends. According to the IMS Climate Atlas (Jerusalem Centre, 1991–2020), the region experiences an average of only **57.5 rainy days** per year (> 0.1 mm), with a median rainfall of **0 mm** for the entire summer period (June–September) [4].

A standard neural network optimizing for Mean Squared Error (MSE) struggles with such long-tailed distributions. Facing a $> 98\%$ "No-Rain" probability, the model often converges to a trivial local minimum where it predicts zero rain for every timestamp [3]. While this strategy yields high statistical accuracy, it results in a Critical Success Index (CSI) near zero, failing to capture the very storm events the system is designed to predict.

Mitigation: We address this by implementing a **Class-Weighted Loss Function**. As proposed in the SaTformer architecture, we re-weight the loss function to assign significantly higher penalties to prediction errors occurring during "Rain" events compared to the dominant "No-Rain" class [3]. This forces the optimizer to prioritize the physics of storm formation over the statistics of clear skies.

2. Data Discontinuity and Multi-Source Synchronization

Challenge: Synchronizing the EUMETSAT satellite feed with the IMS ground station network is prone to inherent data gaps. Ground stations frequently suffer from sensor outages, while satellite feeds occasionally drop frames due to transmission errors [5, 2]. Training on such misaligned or zero-filled data risks "modal collapse," where the model learns to simulate missing data artifacts rather than actual weather physics.

Mitigation: To ensure data integrity, we apply a strict two-tier cleaning protocol:

- **Tier 1 (Temporal Gap Filling):** For small ground station discrepancies (< 30 min), we employ linear gap filling to align the 10-minute IMS sensor cycle with the 15-minute Satellite timestamp.
- **Tier 2 (Rejection):** For significant outages (> 30 min) or any missing satellite frame, the entire training sequence is discarded. We prioritize data purity over quantity to ensure every sample represents a valid physical state.

6 Solution: The Cascaded Dual-Supervision Architecture

6.1 System Overview

We propose a **Cascaded Spatiotemporal Network** designed to enforce physical causality in weather prediction. Unlike standard video prediction models that map past pixels directly to future pixels (treating weather as a simple movie), the proposed system will operate in a sequential, auto-regressive cascade. This design ensures the model implicitly learns the forces driving the weather system before attempting to predict the storm's future trajectory.

The inference process will proceed in two distinct stages for each time step:

1. **Stage 1 (The Drivers):** The model will first predict the invisible thermodynamic state of the atmosphere-specifically, the Surface Wind and Temperature fields.
2. **Stage 2 (The Manifestation):** The model will fuse these predicted physical drivers with the original Latent Features from the encoder to condition the generation of the visible Cloud Structure and the resulting Rain Intensity.

This process is designed to repeat for each time step, with the outputs of Stage 2 feeding back into the input for the next time interval, creating a continuous, physics-informed forecast loop.

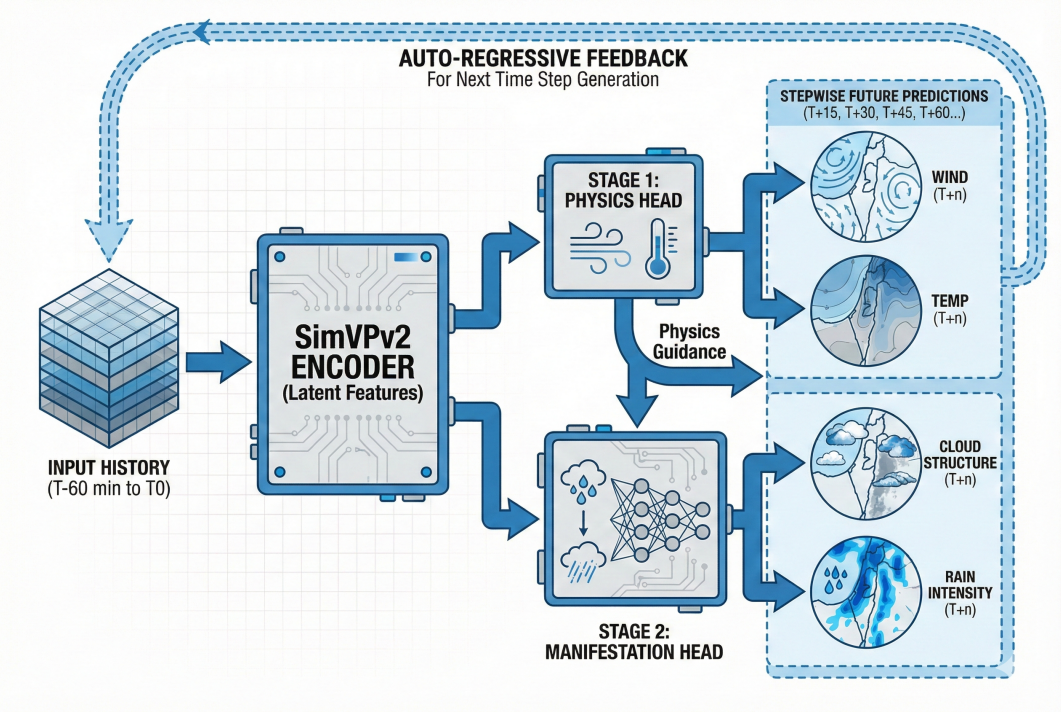

Figure 4: **Proposed System Architecture.** The diagram illustrates the physics-informed cascade. The model first resolves thermodynamic drivers (Wind/Temp) in Stage 1, which then condition the prediction of storm manifestation (Clouds/Rain) in Stage 2. The dashed feedback loop indicates the auto-regressive mechanism used to generate multi-step future sequences (e.g., T+15, T+30, etc.). *(Image generated using Google Nano Banana Pro).*

6.2 The Visual Body (SimVPv2 Encoder)

The foundation of the architecture is the SimVPv2 (Simple Video Predictor v2) Encoder. This component acts as the primary feature extractor, responsible for compressing the high-dimensional raw input into a compact latent representation.

The encoder will process a 5-dimensional input tensor of shape (B, T, C, H, W), where:

- B: Batch Size.
- $T = 4$: The sequence length (past 60 minutes).
- $C = 3$: The input channels (Satellite IR, Water Vapor, Topography).
- $H, W = 256$: The spatial resolution.

Functionally, the encoder is designed to utilize a stack of standard Conv2d layers followed by the specialized Gated Spatiotemporal Attention (gSTA) modules. Unlike heavier Transformer encoders that compute $O(N^2)$ attention maps, gSTA achieves linear complexity by separating spatial and temporal attention. This ensures the model can extract complex motion patterns (e.g., vortex rotation, cloud expansion) while remaining lightweight enough to run on standard GPU hardware.

The output of this stage is the Latent Feature Tensor, which preserves the spatiotemporal context required by both downstream heads.

6.3 Stage 1: The Physics Drivers

The first stage of the cascade explicitly predicts the atmospheric forces driving the weather system, ensuring the model understands the physical context before it attempts to predict cloud movement.

- **Input:** The Latent Spatiotemporal Features output by the SimVPv2 Encoder.
- **Architecture:** A lightweight convolutional decoder designed to project the high-level latent features back into the spatial domain (256×256).
- **Output:** A 2-Channel Dense Tensor representing the thermodynamic state:
    1. **Surface Wind Speed ($\hat{Y}_{wind}$):** Identifying the steering currents.
    2. **Surface Temperature ($\hat{Y}_{temp}$):** Identifying the convective potential.

The Sparse Supervision Strategy: A critical challenge in this stage is that ground truth data exist only at specific station points, while the model must output a complete map. To resolve this, the system will utilizes a Masked Loss Function. During training, the error will be calculated only at the pixels corresponding to active IMS stations.

CNNs are naturally good at recognizing patterns in an image. We use this ability to intelligently 'fill in the blanks' between our scattered weather stations. The model looks at the known data and generates a smooth, continuous map of wind and temperature for the areas in between.

6.4 Stage 2: The Cascade Fusion

This stage functions as the critical integration point that will enforce physical consistency within the model. Unlike standard architectures that treat all variables independently, the proposed system is designed to explicitly condition the visual forecast on the physical drivers predicted in the previous step.

The fusion process will utilize a Channel-Wise Concatenation mechanism:

$$Input_{manifestation} = Z \oplus \hat{Y}_{wind} \oplus \hat{Y}_{temp} \tag{1}$$

Where:

- Z**:** Is the latent feature tensor from the Encoder (containing the visual history of the clouds).
- $\hat{Y}_{wind}, \hat{Y}_{temp}$**:** Are the predicted physical driver maps from Stage 1.
- $\oplus$**:** Represents concatenation along the channel dimension.

Why this matters: By fusing these tensors, the Stage 2 "Manifestation Head" will receive a composite input that contains both the appearance of the storm (from Z) and thermodynamic state (from Stage 1). This ensures that the network "sees" where the wind is pushing the air mass before it decides where to generate the next frame of rain pixels.

6.5 Stage 2: The Transformer Rain Head (SaTformer Logic)

To bridge the gap between simple regression and high-fidelity event detection, the final output head will utilize a Transformer-based Classification mechanism, adopting the classification logic of the SaTformer architecture.

Rather than predicting a single continuous number (regression) or using broad categories, we will segment the precipitation range into 64 Intensity buckets. This transforms the difficult regression task into a classification problem, where the model predicts the probability of the rain falling into a specific narrow range.

- **Input:** The fused physical-visual context tensor ($Input_{manifestation}$).
- **Mechanism:** The model outputs a probability distribution over 64 classes (bins) for each spatial location.
- **Classes Examples (Total 64 Classes):**
  - **Class 0:** Zero Precipitation (0.0 mm/hr)
  - **Class 1:** Trace ($0.0 - 0.2$ mm/hr)
  - **...**
  - **Class 24:** Moderate Intensity ($4.8 - 5.0$ mm/hr)
  - **...**
  - **Class 63:** Extreme Intensity (> 50.0 mm/hr)

The Final Output (Tabular Readout): The system will extract the most probable class for specific (x, y) coordinates to generate a precision forecast table:

Location	Wind (m/s)	Temp (°C)	Precipitation Bin
Tel Aviv	12.2	18.1	**Class 58** (High: 25-28 mm/hr)
Haifa	5.8	19.4	**Class 0** (Dry)
Jerusalem	8.1	16.2	**Class 24** (Mod: 4.8-5.0 mm/hr)
Eilat	2.1	24.5	**Class 0** (Dry)

Table 1: **Target Output Format.** The system classifies rain intensity into 64 high-resolution bins, providing specific ranges for decision support.

6.6 Loss Functions

To train the network effectively on the sparse ground-truth data, the optimization process will employ a Multi-Objective Strategy focused strictly on physical accuracy.

The total loss function is defined as:

$$L_{total} = \lambda_{visual}\mathcal{L}_{cloud} + \lambda_{drivers}\mathcal{L}_{thermo} + \lambda_{rain}\mathcal{L}_{precip} \tag{2}$$

Where the components are defined as follows:

1. Visual Reconstruction Loss ($\mathcal{L}_{cloud}$)

This term acts as the "Dense Supervisor." It compares the predicted cloud structure against the actual future satellite frame over the entire 256×256 grid.

$$\mathcal{L}_{clouds} = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(Y_{true\_img}^{(i,j)} - \hat{Y}_{pred\_img}^{(i,j)} \right)^2 \tag{3}$$

- **Mechanism:** The loss calculates the squared difference between the predicted and actual pixel intensities across the entire 256×256 grid, providing dense supervision for the entire spatial domain.
- **Purpose:** This acts as a physical constraint on the model, requiring it to simulate realistic atmospheric motion where cloud masses evolve via physically valid rotation, expansion, and fluid flow.

2. Thermodynamic Driver Loss ($\mathcal{L}_{thermo}$)

This term optimizes the Stage 1 "Physics Head" (Wind and Temperature). Since these are continuous variables measured only at specific IMS station coordinates, we apply a Masked Mean Squared Error (MSE).

$$\mathcal{L}_{thermo} = \frac{1}{N_{active}} \sum_{(x,y) \in Mask} (Y_{station}^{(x,y)} - \hat{Y}_{pred}^{(x,y)})^2 \tag{4}$$

- **Mechanism:** The loss is calculated only at the pixel coordinates (x, y) corresponding to active stations.
- **Purpose:** This grounds the model in reality. By tying its internal features to actual sensor data, we force it to learn real weather patterns-like a western wind picking up speed-instead of just looking at abstract visual shapes.

3. Precipitation Classification Loss ($\mathcal{L}_{class}$)

This term optimizes the Stage 2 "Transformer Head" (Rain). consistent with the SaTformer logic, we treat rainfall as a classification problem (64 Intensity Buckets) rather than regression. We utilize a Class-Weighted Cross-Entropy Loss to handle the extreme data imbalance.

$$\mathcal{L}_{class} = -\sum_{c=0}^{63} w_c \cdot y_{true,c} \cdot \log(\hat{p}_c) \tag{5}$$

- **Mechanism:** We compare the predicted probability distribution ($\hat{p}$) against the actual rain intensity bucket recorded at the station.
- **Event Weighting (w_c):** To solve the "Zero-Inflation" problem (where 99% of data is "No Rain"), we assign significantly higher weights to the rain classes ($w_{rain} \gg w_{dry}$). This ensures the model is heavily penalized for missing a storm event (False Negative), while penalties for False Alarms on clear days are kept low.

7 Evaluation and Success Metrics

To rigorously assess the performance of the Horizon Forecast system, we define a comprehensive evaluation framework that addresses the specific properties of both continuous thermodynamic variables and stochastic precipitation events.

7.1 Evaluation Metrics

Since the model utilizes a multi-head architecture with different output types, we employ distinct metrics for each meteorological variable.

1. Continuous Variables (Wind & Temperature)

For the Stage 1 Physics Drivers, we utilize standard regression metrics to quantify the deviation between the predicted values and the actual IMS station measurements.

- **Root Mean Square Error (RMSE):**

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{6}$$

Justification: RMSE penalizes large errors more heavily than small ones. In weather safety, being off by 20 m/s (missing a hurricane) is significantly worse than being off by 2 m/s ten times.

2. Categorical Variables (Precipitation)

For the Stage 2 Rain Classifier, standard "Accuracy" is a misleading metric due to the class imbalance (a model predicting "No Rain" forever would achieve 98% accuracy). Instead, we employ **Meteorological Skill Scores** derived from a Confusion Matrix.

For the following equations, we define:

- H: Hits (Correctly predicted rain)
- M: Misses (Failed to predict rain)
- FA: False Alarms (Predicted rain but it was dry)
- CN: Correct Negatives (Correctly predicted dry)

We evaluate these scores at multiple intensity thresholds (e.g., $r > 0.1$ mm/hr, $r > 5.0$ mm/hr):

- **Critical Success Index (CSI):** Also known as the "Threat Score," this is the primary metric for rare event forecasting.

$$CSI = \frac{H}{H + M + FA} \tag{7}$$

Target: A CSI > 0.5 for rain is generally considered state-of-the-art for nowcasting[7].

- **Heidke Skill Score (HSS):** A fractional score that compares the model's accuracy against that of a random guess[11].

$$HSS = \frac{2(H \cdot CN - M \cdot FA)}{(H + M)(M + CN) + (H + FA)(FA + CN)} \tag{8}$$

Justification: This ensures the model is statistically skillful and not simply exploiting the climatological probability of dry weather (where a "Always No-Rain" model would get high accuracy but 0.0 HSS).

3. Visual Quality Metrics (Cloud Structure)

To assess the perceptual quality of the generated cloud imagery (Stage 2 Visual Output), we employ the Structural Similarity Index (SSIM). Unlike MSE, which calculates absolute pixel error, SSIM quantifies the degradation in structural information (texture and sharpness), which is critical for identifying distinct storm cell boundaries.

- **Structural Similarity Index (SSIM):**

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{9}$$

  Where the components are defined as:

  - μ_x, μ_y: The average luminance (brightness) of the Ground Truth (x) and Prediction (y).
  - σ_x^2, σ_y^2: The variance (contrast) of the images. This penalizes "blurry" predictions.
  - σ_{xy}: The covariance (structural correlation) between x and y.
  - C_1, C_2: Small constants to prevent division by zero.

  Justification: A SSIM score closer to 1.0 indicates that the model preserves the sharp edges and complex textures of clouds. This validates that the model is not suffering from the "blurring" effect common in standard regression models.

7.2 Testing Plan

The testing phase is designed not only to benchmark performance but to validate the core "Physics-First" hypothesis of the research.

Phase 1: Baseline Comparison

We will compare the Horizon Forecast model against three distinct benchmarks to establish relative improvement and architectural validity:

1. **Persistence Baseline (Naive):** A trivial model that assumes "The weather in 15 minutes will be exactly the same as now." This serves as the absolute minimum bar; if the deep learning model cannot beat this, it has failed to learn temporal dynamics.
2. **Full SaTformer (State-of-the-Art):** Since our architecture integrates a similar Transformer-based classification head, comparing against the original SaTformer is essential to validate our Hybrid Backbone strategy.

Phase 2: Component Contribution Analysis

To validate the research hypothesis-that explicit physical modeling enhances forecast accuracy, we conduct a Systematic Isolation Test:

- **Experiment:** We will sever the connection between Stage 1 and Stage 2 (removing the Wind/Temp embeddings from the Rain Head input).
- **Expectation:** We expect to see a degradation in the CSI Score for the "Rain" variable. This would prove that the model relies on the thermodynamic context to correctly locate storm cells.

Phase 3: Case Study Analysis

To complement the statistical metrics, we will perform a qualitative analysis on selected historical weather events from the Test Set (2024-2025).

Rather than relying solely on aggregate scores, we will generate the full tabular forecast for specific timestamps and directly compare the model's predictions against the raw IMS station logs. This side-by-side inspection serves to verify the model's reasoning capabilities in real-world scenarios, specifically checking if the system can successfully distinguish between a non precipitating cloud cover and actual precipitation events across different weather conditions.

8 Usage of AI Tools

In accordance with academic integrity guidelines, we declare the following usage of Generative AI tools during Phase A:

- **Large Language Models (Gemini 3 Pro):** Used as a "Technical Advisor" for brainstorming the "Cascaded Dual-Supervision" architecture, refining the mathematical formulation of the Loss Functions, and generating the LaTeX templates for this report.
  - **Example Prompt:** "Explain exactly how SSIM function works (every variable) and convert the function into latex code for overleaf. "
- **Image Generation (Google Nano Banana):** Used to generate the Project Lifecycle Gantt chart (Figure 3) and visualization diagrams to illustrate the system architecture concepts.
  - **Example Prompt:** "Generate a timeline (Gantt) for our project. Divide it into three phases: Phase A (Design & Literature Review), Phase B (Data Engineering), and Phase C (Model & Evaluation). Use icons to match the phase name."
- **Validation:** All AI-generated concepts were manually verified against the official documentation and references.

References

[1] Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.

[2] EUMETSAT. Meteosat second generation (msg) seviri level 1.5 image data. https://data.eumetsat.int, 2025. Accessed: 2025-12-27 via EUMETSAT Data Store API.

[3] Levi Harris and Tianlong Chen. A space-time transformer for precipitation forecasting, 2025.

[4] Israel Meteorological Service. Climate atlas of israel: Precipitation and climate zones. https://ims.gov.il/en/ClimateAtlas, 2025. Accessed: 2026-01-01.

[5] Israel Meteorological Service. Israel meteorological service (ims) data portal: Automated station network data. https://ims.gov.il/en/data_gov, 2025. Accessed: 2026-01-01.

[6] Gavin D. Madakumbura, Chad W. Thackeray, Jesse Norris, Naomi Goldenson, and Alex Hall. Anthropogenic influence on extreme precipitation over global land areas seen in multiple observational datasets. *Nature Communications*, 12(1):3944, 2021.

[7] Alessandro Mazza, Andrea Antonini, Samantha Melani, and Alberto Ortolani. A radar-based fast code for rainfall nowcasting over the tuscany region. *Remote Sensing*, 17(14), 2025.

[8] Jimeng Shi, Azam Shirali, Bowen Jin, Sizhe Zhou, Wei Hu, Rahuul Rangaraj, Shaowen Wang, Jiawei Han, Zhaonan Wang, Upmanu Lall, Yanzhao Wu, Leonardo Bobadilla, and Giri Narasimhan. Deep learning and foundation models for weather prediction: A survey, 2025.

[9] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. Convolutional lstm network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[10] Cheng Tan, Zhangyang Gao, Siyuan Li, and Stan Z. Li. Simvpv2: Towards simple yet powerful spatiotemporal predictive learning. *IEEE Transactions on Multimedia*, 27:5170–5184, 2025.

[11] Huijun Zhang, Yaxin Liu, Chongyu Zhang, and Ningyun Li. Machine learning methods for weather forecasting: A survey. *Atmosphere*, 16(1), 2025.